

Relatório de Auditoria de Segurança — Ipsis

Consulta e geração de peças para advocacia criminal · Next.js 15 + Supabase

Data da auditoria	06/09/2026
Revisão auditada	branch acesso/bloco-2, commit 5e4e1cb
Achados	1 baixa · 3 informativa
Pontos fortes verificados	17 controles conferidos e aprovados
Correção aplicada	09/09/2026 — migration 0026, conferida no banco e com a suíte de navegador
Situação	4 achados, nenhum crítico, alto ou médio. O único com severidade foi corrigido nesta passagem, pela migration 0026; os outros três são informativos. Os nove de 02/09 estão fechados e foram reconferidos na fonte, não no relatório anterior.
Método	Leitura de código, migrations e CI, somada a teste contra as portas reais: PostgREST com a chave pública do bundle, HTTP contra o build de produção, e RLS exercitada com um segundo sub.

Escopo auditado

- `src/` — middleware, seis route handlers, libs de auth, busca, consulta, peça, decretos e vigília, e os componentes de tela.
- `supabase/migrations/` — 26 migrations, e o estado aplicado no banco de produção.
- `coletores/` e `scripts/` — coletores em Python e scripts de lote em TypeScript.
- `.github/workflows/`, `vercel.json`, `next.config.mjs`, `playwright.config.ts` e `e2e/`.
- Histórico do git (172 commits), bundle compilado de um build limpo, e os dados guardados: URLs, textos de lei e o acervo Vade Mecum em disco.

Nota metodológica. As cinco categorias pedidas foram primeiro traduzidas para os mecanismos que esta stack de fato usa, antes de qualquer busca. O isolamento de inquilino aqui é Row Level Security no Postgres, não middleware de tenant nem filtro manual por `usuario_id` — e é por isso que uma query sem `where usuario_id` pode estar correta. Não existem papéis no produto, então a categoria de permissão no navegador foi lida como “gate que a aplicação aplica e o dado não repete”. O mapeamento completo, categoria por categoria, está na página seguinte.

Stack detectada e mapeamento das categorias

Cada categoria da auditoria foi adaptada ao equivalente real desta stack.

Linguagem / runtime	TypeScript 5.9 sobre Node 22; Python 3.12 nos coletores de lote.
Framework	Next.js 15 (App Router), React 19, Server Components e route handlers.
Persistência	Supabase (Postgres + pgvector). Sem ORM: PostgREST via supabase-js (.from() / .rpc()) em runtime; driver `postgres` em conexão direta só em scripts/.
Isolamento de inquilino	Row Level Security no Postgres, ancorada em auth.uid(). Não há middleware de tenant nem filtro manual por usuario_id nas queries — a RLS É o mecanismo, e foi testada como tal.
Autenticação	Supabase Auth (e-mail e senha), usuário único, sessão em cookie via @supabase/ssr; decisão de acesso por getUser() no middleware, no layout de (app) e dentro dos route handlers sensíveis.
Frontend	React 19 + Tailwind v4. Nenhum motor de template server-side de HTML.
Deploy / CI	Vercel (vercel.json, dois crons) e GitHub Actions (verificacao.yml e vigilia.yml). Sem Docker, Helm ou Terraform no repositório.

Como cada categoria foi procurada

1. Banco sem tranca

O equivalente da stack é a RLS do Postgres. Conferidas as 26 migrations e, sobre o banco de produção, o estado REALMENTE aplicado: **relrowsecurity** de toda tabela, o texto de cada policy com papel e condição, e os grants de escrita por papel. O isolamento foi então **testado**, assumindo o papel authenticated com um sub diferente — não inferido do SQL.

2. Permissão definida no navegador

O projeto não tem papéis — isAdmin, canEdit e role não existem em src/. A categoria foi lida como “gate que o app aplica e o dado não”: cada função privilegiada foi consultada por has_function_privilege para anon, authenticated e service_role, e a chave publishable do bundle foi usada contra o PostgREST para ver o que ela alcança na prática.

3. IDOR

Percorridos os seis route handlers e as 41 chamadas .from() / .rpc() de src/. Para as três tabelas de dado de pessoa, um segundo usuário autenticado tentou LER, ALTERAR e APAGAR a linha do primeiro pelo id — o IDOR clássico —, dentro de transação com rollback.

4. Chaves expostas

Varredura por regex nos arquivos versionados, nos 172 commits do histórico, no bundle compilado depois de um build limpo e nos arquivos de ambiente; conferência dos defaults de leitura de variável nos dois runtimes e da interpolação de entrada nos workflows.

5. Inputs sem tratamento (XSS)

Busca por dangerouslySetInnerHTML, innerHTML, eval, new Function, srcdoc e document.write; auditoria dos 13 href dinâmicos; e — o mais forte — varredura dos DADOS: as 3.730 URLs guardadas no banco, os 32.086 textos e os 11 MB de HTML do acervo que alimenta o único sink.

Onde a categoria não se aplica a esta stack

Multi-inquilino: organização, workspace, equipe	Não existe conceito de organização ou equipe no schema. O isolamento pedido pela categoria 1 é por usuário, e está implementado por RLS — e desta vez foi testado com um segundo sub, não só lido. Não há “query de listagem que esqueceu o tenant” porque não há tenant: a listagem sem filtro explícito é correta por construção, já que a policy o aplica no banco.
Papéis, admin, gestão de usuários	isAdmin, canEdit e role não existem em src/ — verificado por busca. Não há o par “UI esconde / servidor não confere” da categoria 2 na forma clássica. Ela foi aplicada como “gate do app não espelhado no dado”, e o caso que existia — o teto de gasto concedido a anon — está fechado e foi reconferido.
ORM e SQL montado por concatenação	Não há ORM. O acesso é PostgREST parametrizado e funções SQL declaradas. Os dois pontos que montam filtro com texto do usuário removem os curingas antes (clientes.ts:184, historico.ts:247), e os ids herdados do fio passam por regex estrito. Não há superfície de injeção de SQL.
HTML de e-mail e motor de template no servidor	O projeto não envia e-mail: quem envia é o servidor de Auth do Supabase, a partir de templates do painel. Nenhuma rota renderiza HTML a partir de string. A metade “backend” da categoria 5 não tem onde acontecer.
Docker, Helm, Terraform	Não existem no repositório. A superfície de deploy é vercel.json e dois workflows do GitHub Actions, ambos auditados.

Fora do escopo, e dito de propósito

- “Secure password change” no painel do Supabase: o endpoint público /auth/v1/settings não o expõe, então ele não é afirmável de fora. Vira o achado informativo nº 4.
- Teste de penetração com ferramenta ofensiva. O que se fez foi exercitar as portas reais — PostgREST com a chave pública, HTTP contra o build de produção, RLS com um segundo sub.
- Revisão jurídica do conteúdo. A auditoria é de acesso, segredo e injeção.

Resumo executivo

4 achados e 17 controles verificados como corretos.



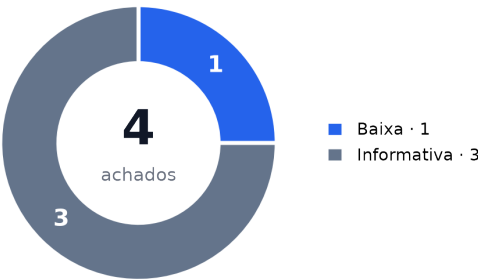
Veredito. Nenhum achado crítico, alto ou médio — e a diferença para a passagem de 02/09 é que desta vez o isolamento não foi só lido: foi **testado**. Um segundo usuário autenticado, com sub diferente, viu zero linha nas quatro tabelas de dado de pessoa e teve UPDATE e DELETE por id alheio recusados. A chave pública do bundle, contra o PostgREST, não lê nada de usuário e não escreve em lugar nenhum.

Os nove achados da primeira passagem estão fechados. Os dois de severidade alta foram reconfirmados na fonte: **disable_signup: true** no servidor de Auth, e os dois tetos de gasto sem EXECUTE para anon e authenticated.

O único achado com severidade era de **defesa em profundidade que envelheceu**: a migration 0004 escreveu “nenhuma escrita para anon” e executou o revoke sobre as tabelas que existiam naquele instante. Sete migrations depois, dez tabelas voltaram a ter grant amplo pelo default do Supabase. A RLS cobre tudo que é alcançável hoje — **menos TRUNCATE, que não passa por política nenhuma**.

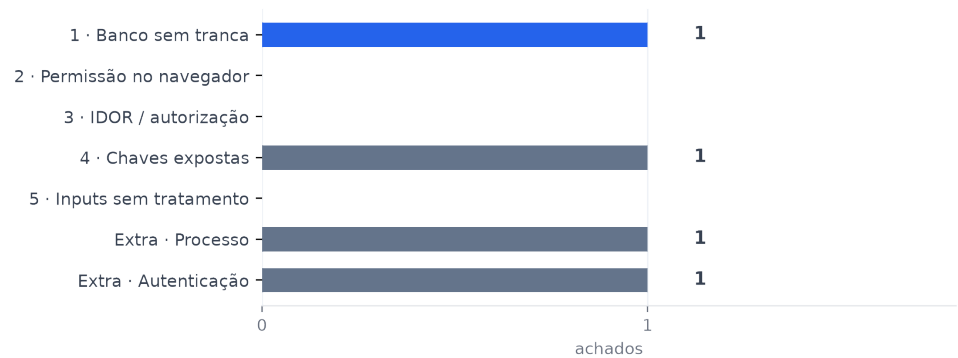
Ele foi corrigido nesta passagem, pela migration 0026: 44 linhas de escrita para anon e authenticated viraram 9, TRUNCATE saiu de todas as tabelas e tabela nova deixa de nascer com grant amplo. Fica registrado que **a correção que este relatório recomendava estava errada**: repetir o revoke em bloco de 0004 derrubaria a agenda de clientes, o histórico e o botão de marcar como conferido, porque RLS decide quais linhas e não se o papel pode escrever. Foi o ensaio em transação com rollback que separou a recomendação plausível da correta — ver o achado 1.

Distribuição por severidade



Achados por categoria

A cor de cada barra é a da severidade mais grave dentro da categoria.



Pontos fortes verificados

O que foi conferido e está correto — a prova de cobertura da auditoria.

01. RLS ligada em 20 de 20 tabelas, conferida nas migrations E no banco aplicado

As 26 migrations declaram ``enable row level security`` para as 20 tabelas que criam — zero divergência entre o que se cria e o que se protege. Contra o banco de produção, a consulta a `pg_class` por `relrowsecurity` devolve nenhuma tabela sem RLS no schema public.

02. O isolamento entre usuários foi TESTADO, não inferido do SQL

Assumindo o papel ``authenticated`` com um ``sub`` diferente do dono real, dentro de transação com `rollback`: `SELECT` em clientes, conversas, conversa_trocas e perfil devolveu 0 linhas nas quatro; `UPDATE` e `DELETE` em clientes pelo id do dono afetaram 0 linhas; `DELETE` em conversas pelo id do dono afetou 0 linhas. Contagens antes e depois idênticas — 1 cliente, 69 conversas. É o IDOR clássico tentado de propósito, e recusado.

03. A chave pública do bundle não alcança dado de pessoa, e isso foi exercitado

Com a chave publishable extraída do próprio bundle, contra o PostgREST: `GET` em clientes, conversas, conversa_trocas e perfil devolve ``[]`` nas quatro. `POST` em clientes e conversas devolve 42501 com “new row violates row-level security policy”. `PATCH` em dispositivos — o texto legal, que a decisão nº 1 protege — devolve 42501. `DELETE` em conversas devolve 204 com zero linhas removidas, confirmado por contagem.

04. Os dois tetos de gasto estão fechados para quem não é o servidor

`has_function_privilege` devolve false para anon e para authenticated, e true só para service_role, nas duas funções ``security definer`` do schema — `consome_uso_ilm` e `consome_uso_busca`. Pelo PostgREST com a chave pública, as duas devolvem 42501 “permission denied for function”. Era o achado de severidade alta da passagem anterior, e está fechado.

05. As duas funções ``security definer`` têm `search_path` fixo

``search_path=public, pg_temp`` nas duas, o que impede sequestro de tabela por schema no caminho do chamador. As outras sete funções do schema são `security invoker` e ficam sujeitas à RLS de quem as chama.

06. Não existe caminho de SQL arbitrário para a chave pública

Sete nomes usuais de RPC de execução (`exec`, `exec_sql`, `execute_sql`, `sql`, `query`, `run_sql`, `pg_execute`) foram tentados contra o PostgREST: todos 404. As únicas funções que anon executa são as sete de leitura do corpus — `busca_hibrida`, `busca_decretos`, `saude`, `norm`, `norm_arr`, `arr_join` e `filtra_artigos_do_precedente`.

07. Os seis route handlers têm, cada um, o porteiro que lhes cabe

`/api/consulta/aovivo` e `/api/peca/[casold]` chamam `usuarioAtual()` no próprio handler; `/api/vigilia/coletar` exige `Authorization: Bearer $CRON_SECRET` e recusa com 503 quando o segredo não está configurado; `/api/health` e `/api/busca` são públicas por decisão escrita; `/auth/confirmar` troca um código de uso único emitido pelo servidor de Auth.

08. O contorno do matcher que a passagem anterior encontrou está fechado

Contra ``next start``: `/api/peca/caso.txt` devolve 307 para `/login`, e não mais 200 — era o caminho que pulava o middleware por terminar em extensão excluída. `/consulta`, `/clientes`, `/configuracoes`, `/api/consulta/aovivo` e `/api/peca/<id>` também devolvem 307 sem sessão.

09. Cinco cabeçalhos de segurança, com nonce novo a cada requisição

Contra o build de produção: `X-Content-Type-Options`, `Referrer-Policy`, `X-Frame-Options: DENY`, `Permissions-Policy` e `Content-Security-Policy`. Duas requisições seguidas trouxeram nonces diferentes, o que prova que a política é montada por requisição e não fixada no build.

10. Nenhum segredo no código versionado, no histórico ou no bundle

Os três arquivos que casam o regex de segredo são placeholders: ``.env.example:64`` (SEU-PROJETO:SENHA@), ``.README.md:445`` (`<ref>:<senha>@`) e uma linha de `dados_auditoria.py` que descreve o próprio regex. Nos 172 commits do histórico, nenhum arquivo `.env`, `.pem`, `.key` ou de credencial jamais foi adicionado — só `.env.example`. No bundle de um build limpo, a única ocorrência é código da biblioteca do Supabase testando prefixo de chave; a única chave presente é a `publishable`, pública por construção.

11. Nenhum default inseguro na leitura de variável de ambiente

O único fallback literal em `src/` e `scripts/` é ``.OPENAI_MODEL ?? 'gpt-5.4-mini'``, que é nome de modelo, não credencial. Do lado Python, os quatro `getenv` usam default vazio. Quem falta variável lança no `import` (`lib/supabase.ts`) ou devolve `null` para quem sabe decidir (`lib/servico.ts`).

12. O único sink de XSS recebe HTML saneado, e os bytes foram conferidos

``.dangerouslySetInnerHTML`` aparece uma vez em produção, em `vademecum/[leild]/page.tsx:117`. O HTML vem de `scripts/vademecum.ts:360`, saneado em build por `allowlist` com `sanitize-html 2.17.7` — o patch da passagem anterior está aplicado. Os 11 MB de acervo em disco foram varridos: ZERO ocorrências de `<script>`, `javascript:`, `onerror=`, `onload=`, `onclick=`, `<iframe>`, `<object>`, `<embed>`, `<svg>` ou `<style>`. As tags presentes são só de documento: `p`, `span`, `a`, `br`, `h1-h5`, `li`, `td`, `i`.

13. Zero eval, new Function, innerHTML, srcdoc ou document.write em todo o código

A varredura de `src/`, `e2e/` e `scripts/` não devolve nenhuma ocorrência dos cinco. A prosa gerada pelo modelo é renderizada como filho de JSX, portanto escapada pelo React.

14. Os dados guardados não carregam esquema perigoso nem marcação executável

Isto é mais forte que ler o código, e por isso foi feito: as 3.730 URLs do banco — 1.496 em `decretos_pr`, 894 em `vigilia_alteracoes`, 1.340 em `artigos.fonte_redacao` — casam todas ``.https?:/``, nenhuma com `javascript:` ou `data:`. Os 32.086 textos guardados (3.771 dispositivos e 28.315 blocos de decreto) não têm uma única ocorrência de `<script>`, `javascript:`, `onerror=` ou `onload=`.

15. O workflow que carrega a service role não interpola entrada no shell

Em `vigilia.yml`, ``.inputs.desde`` e ``.inputs.seco`` aparecem apenas sob ``.env:``, nas linhas 131 e 132, e são referenciados como variáveis citadas dentro do script, com validação de formato ISO antes do uso. O job declara ``.permissions: contents: read``. Nenhum segredo está escrito no arquivo — todos vêm de ``.secrets.``.

16. O cadastro público está fechado, confirmado no servidor de Auth

``.GET /auth/v1/settings`` responde ``.disable_signup: true``. Não é a leitura do painel: é o que o servidor de Auth informa a quem pergunta. Era o achado de severidade alta nº 1 da passagem anterior — o que dava sessão `authenticated` a qualquer visitante.

17. As suítes seguem verdes, e são elas que trancam o que erra em silêncio

``.npm run verificar``: 271 asserções em 11 suítes. ``.pytest coletores``: 120. ``.tests/acesso.test.ts`` guarda as três regras que falham sem avisar — o matcher do middleware (conferindo que o regex do teste é literalmente o do arquivo), a lista de rotas públicas, e a normalização da chave do cache de embedding.

Pontos fracos

Os riscos centrais, ditos sem rodeio.

01. A camada de grant não acompanhou o schema

0004 escreveu a regra e a executou sobre as tabelas que existiam naquele instante. Sete migrations depois, dez tabelas voltaram a ter grant amplo por default do Supabase. A RLS cobre o que é alcançável hoje; o TRUNCATE ela não cobre, e a única razão de isso não doer é que falta caminho, não tranca.

02. Um controle de que a segurança depende não é verificável pela auditoria

“Secure password change” mora no painel e não aparece em nenhum endpoint público. O produto aposta nele e não tem como demonstrá-lo — o que é diferente de estar errado.

03. A fonte de verdade que guarda as exigências de painel existe em duas cópias

AGENTS.md e CLAUDE.md têm 2.573 linhas cada e diferem em duas. É onde vive a exigência de manter o cadastro fechado — a correção do achado mais grave da passagem anterior.

Achados — visão de tabela

Severidade, arquivo:linha e descrição. O detalhe de cada um vem depois.

Sev.	Situação	#	Arquivo : linha	Descrição
Baixa	Resolvido	1	supabase/migrations/0004_rls.sql:22-23 supabase/migrations/0007_conversas.sql (e 0008, 0009, 0012, 0013, 0014, 0018)	anon e authenticated mantinham grants de escrita e TRUNCATE em dez tabelas, e TRUNCATE não passa por RLS 1 · Banco sem tranca
Informativa	Em aberto	2	.github/workflows/verificacao.yml:35-38 .github/workflows/vigilia.yml:69-70 (o contraste)	verificacao.yml não declara `permissions:`, e roda em pull_request 4 · Chaves expostas
Informativa	Em aberto	3	AGENTS.md (2.573 linhas, não versionado) CLAUDE.md (2.573 linhas)	AGENTS.md duplica o CLAUDE.md inteiro, e é lá que moram as exigências de painel Extra · Processo
Informativa	Em aberto	4	CLAUDE.md – tabela de configuração exigida no painel src/components/auth/redefinir-senha.tsx:53-62	“Secure password change” não é verificável de fora do painel Extra · Autenticação

Achados detalhados

Cada achado com evidência, explorabilidade, impacto e condição.

Baixa**Resolvido**

Achado 1 — anon e authenticated mantinham grants de escrita e TRUNCATE em dez tabelas, e TRUNCATE não passa por RLS

Categoria: 1 · Banco sem tranca · Arquivos:

supabase/migrations/0004_rls.sql:22-23 supabase/migrations/0007_conversas.sql (e 0008, 0009, 0012, 0013, 0014, 0018)

Evidência

```
-- 0004_rls.sql:8 a regra declarada
-- Regra: nenhuma policy de insert/update/delete para anon. Nenhuma.

-- 0004_rls.sql:22 o que de fato foi executado
revoke insert, update, delete, truncate on all tables in schema public
from anon, authenticated;
-- ^ ALL TABLES alcança as tabelas que EXISTEM neste instante.
-- conversas (0007), perfil (0008), clientes (0009) e as de
-- 0012/0013/0014/0018 nasceram depois, e o ALTER DEFAULT
-- PRIVILEGES do Supabase devolveu GRANT ALL a elas.

-- estado medido hoje, em information_schema.role_table_grants:
-- clientes anon DELETE,INSERT,TRUNCATE,UPDATE
-- conversas anon DELETE,INSERT,TRUNCATE,UPDATE
-- conversa_trocas anon DELETE,INSERT,TRUNCATE,UPDATE
-- perfil anon DELETE,INSERT,TRUNCATE,UPDATE
-- +6 tabelas anon TRUNCATE
```

Por que é explorável

A migration 0004 escreveu a regra e executou o revoke, mas `on all tables in schema public` é uma fotografia: alcança o que existe no momento em que roda. Sete migrations posteriores criaram tabelas, e o ALTER DEFAULT PRIVILEGES que o Supabase instala concede ALL a anon e authenticated em tabela nova. A camada de grant, que o documento trata como a segunda tranca, não acompanhou o schema. Para INSERT, UPDATE e DELETE a RLS cobre o buraco, e isso foi CONFIRMADO com a chave pública contra o PostgREST: POST em clientes e em conversas devolvem 42501 (“new row violates row-level security policy”), PATCH em dispositivos e em vigilia_alteracoes devolvem 42501, e o DELETE em conversas devolve 204 com ZERO linhas afetadas — contagem conferida antes e depois. **TRUNCATE é a exceção, e é o motivo de este achado existir.** No Postgres, RLS se aplica a SELECT, INSERT, UPDATE, DELETE e MERGE. TRUNCATE não: quem tem o privilégio esvazia a tabela inteira, políticas à parte.

Impacto

Nenhum hoje, e a razão é que falta um caminho, não uma tranca: o PostgREST não expõe verbo de TRUNCATE, e não há função que execute SQL arbitrário — sete nomes usuais (exec, exec_sql, execute_sql, sql, query, run_sql, pg_execute) foram tentados e todos devolvem 404. As únicas funções que anon executa são as sete de leitura do corpus. O risco é do futuro e é assimétrico: no dia em que alguém publicar uma RPC que trunque, ou abrir qualquer via de SQL para o papel anon, `clientes`, `conversas`, `conversa\_trocas` e `perfil` são esvaziáveis sem que policy nenhuma reclame — e são justamente as quatro que guardam dado de pessoa. Perda total, sem erro e sem rastro.

Condição de explorabilidade

Não explorável pela superfície HTTP de hoje. Exige um caminho de execução de SQL para anon ou authenticated que não existe no projeto. É defesa em profundidade que envelheceu — a regra escrita em 0004 vale mais que o estado atual do banco.

O que foi feito

Fechado por `supabase/migrations/0026\_escrita\_do\_dono.sql`, aplicada e conferida no banco: as 44 linhas de escrita para anon e authenticated caíram para 9, TRUNCATE não sobrou em tabela nenhuma, e `npm run e2e` seguiu 24/24 — inclusive os dois casos que cadastram e apagam um cliente e criam uma conversa. **A correção que este relatório recomendava estava errada, e o registro do erro vale mais que o conserto.** A recomendação era repetir o bloco de 0004: revoke de insert/update/delete/truncate para anon E authenticated. Ensaiado em transação com rollback antes de virar migration, ele derruba o produto por duas vias. **(a) RLS decide QUAIS linhas, não SE o papel pode escrever.** As duas checagens são independentes, e a de privilégio vem primeiro: sem o grant de tabela, authenticated recebe `permission denied for table clientes` antes de qualquer policy ser consultada. Em 0004 o revoke em bloco era inofensivo porque não havia escrita de usuário nenhuma — a autenticação nem existia. A frase “é a RLS que autoriza a escrita legítima, não o grant amplo”, que esta mesma passagem escreveu acima, é falsa. **(b) revogar UPDATE de tabela apaga junto o grant por COLUNA.** `vigilia\_alteracoes` concede UPDATE só em `reconferido\_em` e `reconferido\_por`, e é isso que impede “marcar como conferido” de virar “reescrever o link do ato oficial”. O revoke em bloco zera as duas colunas e o botão para de funcionar — sem erro na migration, e sem nada dizendo o motivo. O que 0026 faz, então, é nominal: TRUNCATE sai de todas as tabelas, por ser a única escrita que a RLS não alcança e não ter uso legítimo pelo navegador; INSERT/UPDATE/DELETE saem inteiros de `anon`, que não escreve em lugar nenhum; e de `authenticated` sai só o que `src/` não usa. Sobram as cinco escritas reais do produto, levantadas do código e não estimadas. E a metade que faz a correção durar: `alter default privileges ... revoke`, para o papel `postgres`. Sem ela, 0026 teria a mesma validade de 0004 — a de uma fotografia. Tabela nova passa a exigir grant explícito na própria migration, que é a decisão aparecendo no diff em vez de ser herdada sem ninguém notar.

Informativa

Em aberto

Achado 2 — verificacao.yml não declara `permissions:`, e roda em pull_request

Categoria: 4 · Chaves expostas · Arquivos:

`.github/workflows/verificacao.yml:35-38` `.github/workflows/vigilia.yml:69-70` (o contraste)

Evidência

```
# .github/workflows/verificacao.yml — não há bloco `permissions:`  
on:  
  push:  
    branches: [main]  
  pull_request:  
  
# .github/workflows/vigilia.yml:69 — este declara  
permissions:  
  contents: read
```

Por que é explorável

Sem bloco ``permissions:``, o `GITHUB_TOKEN` do job recebe o conjunto padrão configurado na organização ou no repositório, que pode ser mais amplo que o necessário. O workflow da vigília já declara o mínimo; o de verificação não, e a assimetria é acidental — os dois arquivos foram escritos com meses de diferença.

Impacto

Baixo e indireto. Este workflow não usa segredo nenhum — é o que permite que ele rode com segurança em PR vindo de fork —, e para `pull_request` de fork o GitHub já entrega token somente-leitura. O que se ganha declarando é deixar de depender de uma configuração que mora fora do arquivo e pode mudar sem ninguém notar.

Condição de explorabilidade

Só vira consequência se a configuração padrão do repositório for permissiva e algum passo do job for comprometido — por dependência, por exemplo.

Situação e recomendação

Duas linhas no topo de `verificacao.yml`, iguais às de `vigilia.yml`: `permissions: contents: read`. Nenhum passo do arquivo escreve nada no repositório, então não há o que quebrar.

Informativa

Em aberto

Achado 3 — AGENTS.md duplica o CLAUDE.md inteiro, e é lá que moram as exigências de painel

Categoria: Extra · Processo · Arquivos: AGENTS.md (2.573 linhas, não versionado) CLAUDE.md (2.573 linhas)

Evidência

```
$ diff AGENTS.md CLAUDE.md
685c685
< Codex é superfície de gasto anônima — e a autenticação, quando entrou, não
---
> Claude é superfície de gasto anônima — e a autenticação, quando entrou, não
982c982
< primeira versão trazia `Codex-opus-5` escrito no JSX, e continuou exibindo isso
---
> primeira versão trazia `claude-opus-5` escrito no JSX, e continuou exibindo isso
```

Por que é explorável

São 2.573 linhas duplicadas que diferem em duas — o nome do agente. Não há segredo em nenhum dos dois arquivos, e isso foi conferido. O que faz disto assunto de segurança é o CONTEÚDO: é neste documento que vivem as quatro configurações obrigatórias do painel do Supabase, incluindo “Allow new users to sign up: desligado”, que é a correção do achado de severidade alta da passagem anterior. O projeto escolheu esse documento como o lugar onde a exigência sobrevive ao próximo deploy. Duas cópias significam que ela pode passar a existir em uma e não na outra, e o agente que ler a errada não saberá.

Impacto

Nenhum imediato: hoje as duas cópias dizem a mesma coisa. O risco é de divergência silenciosa — a mesma classe de erro que o `marca.ts` do projeto existe para impedir, e que o próprio documento descreve como “é assim que uma tela fica com o nome antigo”.

Condição de explorabilidade

Não é uma falha explorável; é uma fonte de verdade duplicada.

Situação e recomendação

Duas saídas, e a escolha é de processo, não técnica. A primeira é AGENTS.md virar um arquivo de três linhas apontando para o CLAUDE.md, o que preserva a convenção que outros agentes procuram sem duplicar conteúdo. A segunda é gerar um do outro num passo do `npm run verificar`, que falha se divergirem — a mesma disciplina que `tests/vigilia.test.ts` aplica aos dois runtimes do filtro.

Informativa

Em aberto

Achado 4 — “Secure password change” não é verificável de fora do painel

Categoria: Extra · Autenticação · Arquivos: CLAUDE.md — tabela de configuração exigida no painel src/components/auth/redefinir-senha.tsx:53-62

Evidência

```
GET /auth/v1/settings (endpoint público do Auth)
  disable_signup ..... true    <- verificável, e está fechado
  mailer_autoconfirm ..... true  <- verificável
  external.email ..... true     <- verificável
  secure_password_change . (ausente do payload)

// src/components/auth/redefinir-senha.tsx:54
const { error } = await supabase.auth.updateUser({ password: senha })
//                                     ^ sem senha atual, por desenho do SDK
```

Por que é explorável

O produto depende de um interruptor do painel para fechar a troca de senha sem reautenticação, e o endpoint público de configurações não devolve esse campo. Confirmei os outros três; este não dá para afirmar nem negar de fora. A guarda que existe em código — o cookie `ipsis-recuperacao`, posto por /auth/confirmar e exigido por /redefinir-senha — impede o acidente de quem já está logado abrir a tela, e isso foi verificado no navegador. Ela não impede quem tem um cookie válido e chama o SDK sem passar por tela nenhuma.

Impacto

Se o interruptor estiver desligado, um cookie de sessão obtido por outro caminho permite trocar a senha sem conhecer a antiga — tomada permanente da conta. Se estiver ligado, não há impacto. A auditoria não consegue dizer qual dos dois é o caso.

Condição de explorabilidade

Depende de uma configuração fora do repositório. O dono relatou tê-la ligado; o que falta é evidência que a auditoria possa produzir sozinha.

Situação e recomendação

Conferir uma vez no painel, e depois anotar a data da conferência ao lado da linha na tabela do CLAUDE.md — é o que o projeto já faz com `artigos.conferido\_em` e com o `conferido\_em` dos decretos: quando não dá para medir, registra-se quem olhou e quando. Alternativa mais forte: um caso de e2e que faça login, chame updateUser sem reautenticar e espere a recusa.

Recomendações priorizadas

Por ordem de consequência, não de esforço.

P1 Devolver a camada de grant ao mínimo — feito nesta passagem

- ``0026_escrita_do_dono.sql``, aplicada e conferida: 44 linhas de escrita para anon e authenticated viraram 9, TRUNCATE saiu de todas as tabelas, e o ``alter default privileges`` faz a próxima tabela nascer fechada. ``npm run e2e`` seguiu 24/24. (achado 1)
- O revoke em bloco que este relatório recomendava foi ENSAIADO e recusado: ele derruba clientes, conversas, perfil e o botão de marcar como conferido. RLS decide quais linhas, não se o papel pode escrever — e revogar UPDATE de tabela apaga o grant por coluna junto. O que entrou é nominal, levantado da superfície de escrita real de ``src/``.
- Continua em aberto: somar a asserção ao ``npm run verificar``, na forma que o projeto já usa para o matcher — uma consulta que falhe se anon voltar a ter escrita. Grant é estado do banco, e estado que ninguém observa é estado que volta. 0026 fecha a porta de hoje; só a asserção impede a 0027 de reabri-la.

P2 Fechar as duas frouxidões de processo

- Declarar ``permissions: contents: read`` em `verificacao.yml`, como `vigilia.yml` já faz. Duas linhas, nenhum passo do arquivo escreve no repositório. (achado 2)
- Resolver a duplicação de `AGENTS.md` — apontar para o `CLAUDE.md`, ou gerar um do outro com uma asserção que falhe na divergência. É onde moram as exigências de painel. (achado 3)

P3 Tornar verificável o que hoje é declarado

- Conferir “Secure password change” no painel e anotar a data ao lado da linha na tabela do `CLAUDE.md` — a mesma disciplina de ``conferido_em`` que o projeto aplica ao corpus e aos decretos. (achado 4)
- Opcionalmente, um caso de e2e que faça login, chame `updateUser` sem reautenticar e espere a recusa: transforma uma configuração de painel em asserção que roda.
- Quando a landing page existir, decidir `robots.ts` e `sitemap.ts`: hoje o app inteiro é rastreável por padrão e `/login` e `/cadastro` entram no índice.

Issues para o GitHub

Texto integral em Markdown, pronto para copiar e colar. Cada bloco vai entre os delimitadores.

São 3 issues para 4 achados: os dois informativos de higiene — CI e documentação duplicada — foram agrupados, porque são o mesmo tipo de trabalho e abrir duas seria spam.

--- ISSUE 1 --- Travar por asserção que anon não volte a ter escrita no banco

```
# [Segurança] Travar por asserção que `anon` não volte a ter escrita no banco

**Labels:** `security`, `severidade: baixa`, `banco`, `rls`, `teste`

> **0 achado que originou esta issue já foi corrigido** por
> `supabase/migrations/0026_escrita_do_dono.sql`, aplicada e conferida. 0 que
> sobra — e é o que esta issue pede — é a asserção que impede o problema de
> voltar. Ver o histórico abaixo, que vale como registro.

## 0 que estava errado

`0004_rls.sql` declarava "nenhuma escrita para anon" e executava:

```sql
revoke insert, update, delete, truncate on all tables in schema public
from anon, authenticated;
```

`on all tables in schema public` é uma **fotografia**: alcança as tabelas que
existem no instante em que roda. Dez tabelas criadas depois — `conversas` e
`conversa_trocas` (0007), `perfil` (0008), `clientes` (0009), as da vigília
(0012, 0013), `precedentes_stj` (0014) e as de decretos (0018) — nasceram com
`GRANT ALL` para `anon` e `authenticated`, porque é o que o
`ALTER DEFAULT PRIVILEGES` do Supabase manda fazer. Medido:
**44 linhas** de INSERT/UPDATE/DELETE/TRUNCATE em
`information_schema.role_table_grants`.

Para DML a RLS cobria. **TRUNCATE não**: no Postgres, RLS se aplica a SELECT,
INSERT, UPDATE, DELETE e MERGE, e quem tem o privilégio de TRUNCATE esvazia a
tabela inteira sem que policy nenhuma reclame.

## A correção que esta issue recomendava estava errada

Vale registrar, porque é a parte reaproveitável. A sugestão original era repetir
o bloco de 0004. **Ensaio em transação com rollback, ele derruba o produto:**

**(a) RLS decide QUAIS linhas, não SE o papel pode escrever.** As duas checagens
são independentes e a de privilégio vem primeiro:

```
revoke insert on public.clientes from authenticated;
set local role authenticated;
insert into public.clientes (usuario_id, nome) values (...);
--> ERROR: permission denied for table clientes
```

Em 0004 o revoke em bloco era inofensivo porque não havia escrita de usuário
nenhuma — a autenticação nem existia.

**(b) revogar UPDATE de tabela apaga junto o grant por COLUNA.**
`vigilia_alteracoes` concede UPDATE só em `reconferido_em` e `reconferido_por`
(0012, 0024), e é isso que impede "marcar como conferido" de virar "reescrever o
link do ato oficial". Medido: depois de `revoke update ... from authenticated`,
`information_schema.column_privileges` devolve **zero** coluna — o botão para de
funcionar, sem erro na migration e sem nada dizendo o motivo.

## 0 que 0026 fez

Nominal, a partir da superfície de escrita levantada de `src/`:

```sql
revoke truncate on all tables in schema public from anon, authenticated;
```

```
revoke insert, update, delete on all tables in schema public from anon;
revoke delete on public.perfil from authenticated;
revoke update, delete on public.conversa_trocas from authenticated;

alter default privileges for role postgres in schema public
 revoke insert, update, delete, truncate on tables from anon, authenticated;
...
```

Resultado conferido no banco: **\*\*44 → 9 linhas\*\***, e as 9 são exatamente as cinco escritas reais do produto (`clientes`, `conversas`, `conversa\_trocas`, `perfil` pelo navegador com sessão, mais o UPDATE por coluna de `vigilia\_alteracoes`). TRUNCATE não sobrou em tabela nenhuma. `npm run e2e` seguiu 24/24.

## 0 que falta, e é o pedido desta issue

O `alter default privileges` faz tabela nova nascer fechada, mas **\*\*nada observa o estado\*\***. Grant é estado do banco, e estado que ninguém observa é estado que volta – foi exatamente assim que 0004 envelheceu sem ninguém notar.

Falta uma asserção, na forma que o projeto já usa para o `matcher` do middleware em `tests/aceso.test.ts`: uma consulta que falhe se `anon` voltar a ter qualquer escrita, ou se `authenticated` ganhar escrita fora da lista conhecida.

Ela **\*\*não cabe no `npm run verificar`\*\***, que roda offline e sem segredo por decisão escrita. O lugar natural é junto de `npm run e2e`, que já fala com o Supabase de verdade, ou um `npm run grants` chamado à mão antes de congelar uma versão.

## Critérios de aceite

- [ ] Existe um comando que lê `information\_schema.role\_table\_grants` e falha se `anon` tiver INSERT, UPDATE, DELETE ou TRUNCATE em qualquer tabela de `public`.
- [ ] O mesmo comando falha se `authenticated` tiver escrita **\*\*fora\*\*** da lista esperada – hoje: `clientes` (I/U/D), `conversas` (I/U/D), `conversa\_trocas` (I), `perfil` (I/U). A lista mora no teste, para que ampliá-la seja uma decisão que aparece no diff.
- [ ] O mesmo comando falha se qualquer papel que não seja `service\_role` ou `postgres` tiver TRUNCATE em `public`.
- [ ] A asserção confere também o `pg\_default\_acl` do papel `postgres`, senão ela passa a verde no dia seguinte ao de alguém reconceder o default.
- [ ] Rodar o comando contra o banco de hoje passa, sem ajuste.

--- FIM ISSUE 1 ---

--- ISSUE 2 --- Higiene: permissions no CI e a duplicação do AGENTS.md

```
[Segurança] Higiene: `permissions:` ausente em verificacao.yml e AGENTS.md duplicando o CLAUDE.md
```

```
Labels: `security`, `severidade: informativa`, `ci`, `documentação`
```

Dois itens de higiene agrupados: são o mesmo tipo de trabalho e abrir duas issues seria spam.

---

## Parte A – `verificacao.yml` não declara `permissions:`

### Problema

Sem bloco `permissions:`, o `GITHUB\_TOKEN` do job recebe o conjunto padrão configurado na organização ou no repositório, que pode ser mais amplo que o necessário. O workflow da vigília já declara o mínimo; o de verificação não, e a assimetria é acidental – os dois arquivos foram escritos com meses de diferença.

### Evidência

```
`github/workflows/verificacao.yml:35-38` – não há bloco `permissions:`
```

```
```yaml
on:
```

```
push:
  branches: [main]
pull_request:
  ...
```

`github/workflows/vigilia.yml:69-70` – o contraste

```
...yaml
permissions:
  contents: read
...
```

Impacto

Baixo e indireto. Este workflow não usa segredo nenhum – é o que permite que ele rode com segurança em PR vindo de fork –, e para `pull\_request` de fork o GitHub já entrega token somente-leitura. O que se ganha declarando é deixar de depender de uma configuração que mora fora do arquivo e pode mudar sem ninguém notar.

Correção

Duas linhas no topo de `verificacao.yml`, iguais às de `vigilia.yml`. Nenhum passo do arquivo escreve no repositório, então não há o que quebrar.

Parte B – AGENTS.md duplica o CLAUDE.md inteiro

Problema

São 2.573 linhas duplicadas que diferem em **duas** – o nome do agente. Não há segredo em nenhum dos dois arquivos, e isso foi conferido.

O que faz disto assunto de segurança é o conteúdo: é neste documento que vivem as quatro configurações obrigatórias do painel do Supabase, incluindo **"Allow new users to sign up: desligado"**, que é a correção do achado de severidade alta da auditoria anterior. O projeto escolheu esse documento como o lugar onde a exigência sobrevive ao próximo deploy.

Duas cópias significam que ela pode passar a existir em uma e não na outra, e o agente que ler a errada não saberá.

Evidência

```
...
$ diff AGENTS.md CLAUDE.md
685c685
< Codex é superfície de gasto anônima – e a autenticação, quando entrou, não
---
> Claude é superfície de gasto anônima – e a autenticação, quando entrou, não
982c982
< primeira versão trazia `Codex-opus-5` escrito no JSX, e continuou exibindo isso
---
> primeira versão trazia `claude-opus-5` escrito no JSX, e continuou exibindo isso
...
```

Impacto

Nenhum imediato: hoje as duas cópias dizem a mesma coisa. O risco é de divergência silenciosa – a mesma classe de erro que o `marca.ts` do projeto existe para impedir, e que o próprio documento descreve como **"é assim que uma tela fica com o nome antigo"**.

Correção

Duas saídas, e a escolha é de processo:

1. `AGENTS.md` vira um arquivo de três linhas apontando para o `CLAUDE.md` – preserva a convenção que outros agentes procuram sem duplicar conteúdo;
2. um é gerado do outro num passo do `npm run verificar`, que falha se divergirem – a mesma disciplina que `tests/vigilia.test.ts` aplica aos dois runtimes do filtro.

Critérios de aceite

- [] `verificacao.yml` declara `permissions: contents: read`.
- [] Um PR de teste continua passando nos dois jobs.
- [] `AGENTS.md` deixa de ser cópia integral: ou aponta para o `CLAUDE.md`, ou passa a ser gerado com asserção que falha na divergência.
- [] A tabela de configuração obrigatória do painel existe em **um** lugar canônico, e o outro arquivo aponta para ele.

--- FIM ISSUE 2 ---

--- ISSUE 3 --- Tornar verificável o Secure password change

[Segurança] "Secure password change" não é verificável de fora do painel

Labels: `security`, `severidade: informativa`, `auth`, `verificação`

Problema

O produto depende de um interruptor do painel do Supabase para fechar a troca de senha sem reautenticação, e o endpoint público de configurações não devolve esse campo. Os outros três ajustes obrigatórios são confirmáveis; este não é afirmável nem negável de fora.

Evidência

`GET /auth/v1/settings` – endpoint público do Auth:

```
```
```

```
disable_signup true <- verificável, e está fechado
mailer_autoconfirm true <- verificável
external.email true <- verificável
secure_password_change . (ausente do payload)
```
```

`src/components/auth/redefinir-senha.tsx:53-62`

```
```ts
```

```
const supabase = supabaseNavegador()
const { error } = await supabase.auth.updateUser({ password: senha })
// ^ sem senha atual, por desenho do SDK
```
```

A guarda que existe em código – o cookie `ipsis-recuperacao`, posto por `/auth/confirmar` e exigido por `/redefinir-senha` – impede o **acidente** de quem já está logado abrir a tela, e isso foi verificado no navegador. Ela não impede quem tem um cookie válido e chama o SDK sem passar por tela nenhuma.

Impacto

Se o interruptor estiver desligado, um cookie de sessão obtido por outro caminho permite trocar a senha sem conhecer a antiga – tomada permanente da conta. Se estiver ligado, não há impacto.

A auditoria não consegue dizer qual dos dois é o caso, e é essa a lacuna que esta issue registra.

Sugestão de correção

1. Conferir uma vez no painel: Authentication → Providers → Email → **Secure password change**.
2. Anotar a data da conferência ao lado da linha na tabela do `CLAUDE.md` – é a mesma disciplina de `artigos.conferido\_em` e do `conferido\_em` dos decretos: quando não dá para medir, registra-se quem olhou e quando.
3. Mais forte, e opcional: um caso de e2e que faça login, chame `updateUser({ password })` sem reautenticar e espere a recusa. Transforma uma configuração de painel numa asserção que roda.

Critérios de aceite

- [] O estado do interruptor foi conferido no painel.

- [] A tabela de configuração obrigatória do `CLAUDE.md` traz a data da conferência ao lado da linha.
- [] Opcional: um caso de e2e cobre a recusa de `updateUser` sem reautenticação, e ele falha se o interruptor for desligado.

--- FIM ISSUE 3 ---